
Oliver Hauke

ist Mathematiker und IT-Referent im Kompetenzzentrum „Analyse und Auswertung“ des Statistischen Bundesamtes. Er verantwortet die Weiterentwicklung der technischen Infrastruktur des Forschungsdatenzentrums und berät das Netzwerk empirische Steuerforschung in der effizienten Nutzung der Systeme.

Annette Kristiansen

ist Ökonomin und Referentin im Referat „Unternehmenssteuern“ des Statistischen Bundesamtes. Sie beschäftigt sich mit der Zusammenführung der Unternehmenssteuerstatistiken und betreut das Netzwerk empirische Steuerforschung. Für ihre externe Promotion an der Universität Bremen untersucht sie die technologische Vielfalt multinationaler Unternehmen.

EFFIZIENTE ANALYSE GROSSER FORSCHUNGSDATEN MITHILFE VON R

Oliver Hauke, Annette Kristiansen

🔑 **Schlüsselwörter:** Effiziente Analysen, R-Programmierung, Forschungsdaten, Unternehmenssteuerstatistik, Business-Tax-Panel, Netzwerk empirische Steuerforschung

ZUSAMMENFASSUNG

Das Forschungsdatenzentrum des Statistischen Bundesamtes baut seine technische Infrastruktur gezielt aus, um der Wissenschaft erweiterte Analysemöglichkeiten amtlicher Daten bereitzustellen. Seit November 2025 steht Forschenden exklusiver Zugang zu diesen erweiterten Systemressourcen in R und Stata zur Verfügung. R-basierte Tools – insbesondere Apache Arrow, Parquet und DuckDB – ermöglichen es, gängige Analysen großer Forschungsdatensätze in Echtzeit auszuführen. Am Beispiel des Business-Tax-Panel (2013 bis 2019) werden die erzielbaren Effizienzgewinne bei der Auswertung umfangreicher steuerstatistischer Daten im Rahmen des Netzwerks empirische Steuerforschung demonstriert.

🔑 **Keywords:** *Efficient analysis – R programming – research data – corporate tax statistics – Business Tax Panel – empirical tax research network*

ABSTRACT

The Research Data Centre at the Federal Statistical Office is strategically expanding its technical infrastructure to provide researchers with enhanced analytical capabilities for official data. Since November 2025, researchers have had exclusive access to these extended system resources in R and Stata. R-based tools – particularly Apache Arrow, Parquet, and DuckDB – enable real-time analysis of large research datasets. Using the Business Tax Panel (2013–2019), this paper demonstrates the achievable efficiency gains in analysing large-scale tax statistics within the empirical tax research network.

5

Vergleich der Performanz

5.1 Benchmark-Setup

Um zwischen den in Abschnitt **?@sec-methoden** vorgestellten Datenverarbeitungsmethoden abzuwägen, haben wir deren Performanz in einem systematischen Benchmark-Experiment auf dem SAS-Server des Statistischen Bundesamtes und den neuen OnSite R- und Stata-Servern des FDZ gemessen. Weiterführende Benchmarks sowie der vollständig reproduzierbare Code sind auf [open-code.de](https://www.opencode.de) verfügbar¹.

Herangezogen wurden dafür simulierte Einzeldaten in verschiedenen Größen (0.1%/1%/10% von den insgesamt 80 Mio. Beobachtungen des BTP). Sie wurden aus öffentlich verfügbaren Informationen generiert, um die wesentlichen Strukturen des BTP für unsere Analyse künstlich nachzubilden. Abgrenzend zu den im FDZ bereitgestellten *Datenstrukturfiles* wurden die Testdaten nicht aus echten Mikrodaten abgeleitet. Der simulierte Datensatz ist ausschließlich für technische Testzwecke der nachfolgenden Anwendungsfälle bestimmt:

1. **Fallzahlen** (pro Statistik und Jahr)
2. **Regression** (Einflussfaktoren auf Verlustvorträge)

Um eine zuverlässige Performanz-Messung zu erhalten, wurde jede Datenverarbeitungsmethode dreifach angewandt. Der Vergleich erfolgt anhand der mittleren Messung der folgenden Zielgrößen:

1. **Arbeitsspeicher**-Beanspruchung der Auswertung,
2. **Laufzeit** der Auswertung (real verstrichene Zeit),
3. **Festplattenspeicher** der verwendeten Daten.

Die Zielgrößen sind in dieser Reihenfolge zu priorisieren, da ein übermäßiger Arbeitsspeicherverbrauch zu schwerwiegenden Abbrüchen führen kann, während längere Laufzeiten dennoch zu einem erfolgreichen Ergebnis führen können. Festplattenspeicher ist ausreichend vorhanden, aber unnötige Belegung sollte vermieden werden.

Für unsere Aufgaben muss nicht zwingend die

Abstufungen der Zielgrößen sind für unsere Aufgaben nicht gleichermaßen relevant.

Eine Methode, die die betrachteten Auswertungen in unter 10 Sekunden mit weniger als 4 GB Arbeitsspeicher durchführen kann, wird bereits als optimal bewertet.

¹www.opencode.de/

Weitere Optimierungen sind in diesem Fall nicht erforderlich. Falls bereits die gewünschte Performanz erreicht wurde, rückt die Reduktion des Entwicklungsaufwand in den Vordergrund. Entsprechend sind Aspekte wie die einfache Anwendbarkeit und Nachvollziehbarkeit – selbst für unerfahrene Nutzende – ausschlaggebend.

Wir betrachten in R (Version 4.5.0) die folgenden R-Pakete gemeinsam mit den in Abschnitt **?@sec-methoden** beschriebenen Methoden (Variablenselektion, Datenaufteilung):

Tabelle 1

Versionen betrachteter R-Pakete

Paket	Version
data.table	1.17.8
tidyverse	tba
vroom	tba
arrow	20.0.0.2
duckdb	1.3.2
polars	1.0.1

Wie in Abschnitt **?@sec-infrastruktur** beschrieben war es uns aus organisatorischen Gründen nicht möglich, die neueste Version aller Pakete zu verwenden. Die leichter nutzbaren Variationen {duckplyr} bzw. {tidytable} zu nutzen. Die Package wurden aber wenige Tage nach dem Experiment eingerichtet. Die Syntaxtranslation kann dabei die Performanz verschlechtern, sodass bei {duckdb} bzw. {data.table} zwischen Performanz und Code-Verständlichkeit abgewägt werden muss. {arrow} ist schon länger als {polars} im Statistischen Bundesamt verfügbar, sodass wir darin bereits mehr Erfahrungen einsetzen konnten.

5.2 Baseline

Klassisch werden die betrachteten Auswertungsaufgaben im Statistischen Bundesamt hauptsächlich mit SAS und im FDZ mit Stata gelöst. Um die nachfolgenden Ergebnisse einzuordnen, haben wir die Performanzmessungen für 1% des simulierten BTP durchgeführt (800.000 Beobachtungen) in SAS und Stata durchgeführt:

Der Arbeitsspeicher des FDZ OnSite-Servers reicht nicht

Tabelle 2

Baseline anhand SAS und Stata (1% simuliertes BTP)

Software	Anwendung	Arbeitsspeicher	Laufzeit	Festplattenspeicher
----------	-----------	-----------------	----------	---------------------

aus, um das ganze BTP in Stata zu verarbeiten, sodass zwingend mit selektivem Einlesen gearbeitet werden muss.

5.3 Hauptergebnis für große Datenmengen

Anhand unserer Untersuchung, bietet `{arrow}`+`{parquet}` (bei Bedarf +`{duckdb}`) im Gesamtpaket die eindeutige besten Datenverarbeitungsmethoden für unsere Zwecke.

- › Das geringe Arbeitsspeicherprofil Bedarf keiner weiteren Optimierung und ist in unserer Auswertung nahezu konstant abhängig von der Datenmenge, sodass die Methoden auf noch größere Datenmengen skalieren können. Ansonsten bietet `{duckdb}` und `{polars}` stärkere Optimierung des Arbeitsspeichers.
- › schnellste Laufzeit für große Datenmengen
- › das Parquet Dateiformat reduziert den benötigten Festplattenspeicher massiv.
- › beste Nutzererfahrung
 - › Leicht und verständlich - adaptiert nativ schöne `{tidyverse}` Syntax
 - › Flexibel, selektives Einlesen ist zwingend erforderlich
 - › hohe Stabilität, fehlende Features können durch leicht in `duckdb` oder in den Arbeitsspeicher überführt und dennoch ausgeführt werden
- › kleiner Nachteil: Erfahrung notwendig, da erst Datenaufteilung/selektives Einlesen die maximale Performance erreichen

Unerwarteterweise ist `{polars}` für das simulierte 10% BTP abgeschlagen (unter den schlechtesten Methoden) und für andere Datenmengen optimal schnell.

5.4 Erkenntnisse für kleine Datenmengen

- › Paket für die optimale Laufzeit nicht eindeutig
 - › 1e6: `arrow` > `duckdb` > `DT+SEL` > `polars`
 - › 1e5: `polars` > `duckdb` > `arrow` > `DT+SEL`
 - › 1e4: `polars` > `data.table` > `arrow` > `readr`
- › Dagegen ist das effizienteste betrachtete Datenformat über 1000 Beobachtungen eindeutig Parquet – sowohl in Speicherbedarf als auch in Laufzeit.
- › Datenaufteilung bietet nur high-performance Methoden ab einer gewissen Datenmenge Vorteile. Für kleine Mengen überwiegt der Overhead durch das Lesen mehrerer kleiner Dateien. Klassische Methoden werden durch eine Datenaufteilung nur negativ beeinflusst.

- › Die Ergänzung von selektivem Einlesen ist immer vorteilhaft. High-Performance Methoden können aber bei einer aufgeteilten Datenhaltung darauf verzichten.
- › Von den klassischen Methoden ist `DT+SEL` am effizientesten, aber für unseren größten Anwendungsfall. Ein geschickter DTler kann mit moderaten Datenmengen auch sehr effizient arbeiten. Anfängern ist eher `{arrow}` zu empfehlen.
 - › 5x langsamer als `{arrow}`
 - › `SEL` notwendig, da ohne 3x größer als die Eingangsdaten, dauert 2.2 min anstatt 23 Sec. -> schwierige Anwendung (Entscheidung relevanter Variablen muss vorab getroffen werden unter Beachtung der Arbeitsspeicher Restriktionen)
 - › `SPLIT` verschlechtert `DT` im Gegensatz zu `arrow`
 - › `{data.table}`+`csv` ist die beste "klassische" R-Methode und erreicht maximal 23.58s, in der größten Ordnung wie die nicht optimierte Arrow Performance. Das die beste gemessene Performance für CSV (aufgeteilte Parquet sind doppelt so schnell). D.h. ein `{data.table}` Profi, ist besser als ein schlechter Arrow Programmierer, der aber größeres Potenzial für Verbesserungen hat.
- › `base`+`readr` sind immer weit abgeschlagen

5.5 Ergebnisse

- › › Für `{data.table}` ist die geschickte Programmierung noch viel wichtiger. Arrow wird höchstens langsam und beansprucht viel weniger RAM. Wohingegen `data.table` für den gesamten Datensatz (ohne `Sel`) das 600x an RAM belegt wie Arrow, damit viel schneller Abbricht und den schlechten Programmierer Abbrüche beschert.
 - › ohne Vorauswahl bricht `DT` ab, wenn das gesamte BTP verwendet werden soll. Arrow nur, wenn die Auswertung auch dazu führt, dass wirklich alle Variablen verarbeitet werden.
 - › mit `DT` lässt sich nicht so flexibel/schön arbeiten
- › Arrow verzeiht Fehler leichter als `DT`. Alleine durch die Datenaufteilung (vorgegeben durch die Datenbereitstellung, vorbereitet durch das FDZ), erreichen wir Count Ergebnisse in 11.73s, nur 5s über dem absoluten Optimum.
- › Wenn noch weniger RAM einsetzbar ist, benötigt `Polars` den wenigsten RAM, `DuckDB` nur geringfügig mehr

Tabelle 3

Performanz Messungen für die Fallzahlberechnung anhand 10% des BTP.

Paket	Dateiformat	Optimierung	Arbeitsspeicher	Laufzeit	Festplattenspeicher
{arrow}	Parquet	Datenaufteilung (gleichmäßig)	219.23M	4.65s	45.5G
{duckdb}	Parquet	-	4.48M	5.14s	46G
{arrow}	Parquet	Variablenauswahl	669.55M	7.47s	46G
{arrow}	Parquet	Datenaufteilung (Berichtsjahr)	4.47M	11.73s	44.8G
{arrow}	Parquet	-	219.7M	16.96s	46G
{arrow}	Parquet	-	668.71M	20.61s	46G
{data.table}	CSV	Variablenauswahl	407.68M	23.58s	121.2G
{polars}	Parquet	-	3.13M	1.17m	46G
{data.table}	CSV	-	312.92G	2.21m	121.2G
{data.table}	CSV	Datenaufteilung (Berichtsjahr)	471.61G	5m	97.5G

Tabelle 4

Performanz Messungen für die Regressionsberechnung anhand 10% des BTP.

Paket	Dateiformat	Optimierung	Arbeitsspeicher	Laufzeit	Festplattenspeicher
{arrow}	Parquet	Datenaufteilung (Berichtsjahr)	745.92M	5.19s	44.8G
{arrow}	Parquet	Datenaufteilung (Berichtsjahr)	745.92M	6.54s	44.9G
{dplyr}	CSV	Variablenauswahl	1.42G	1.92m	121.2G

10 %: arrow + aufgeteilte parquet, duckdb, arrow variante
 1 %: polars + ein parquet, duckdb, arrow 0.1 %: polars +
 ein parquet, DT, arrow 0.01%: DT + csv, readr, polars

-> nicht eindeutig. Für kleinere Datenmengen mehr
 Auswahl und das vertraute auszuwählen. Für größere
 Datenmengen klare Empfehlung Arrow, w.g. einfacher An-
 wendung, Stabilität (ggü. Nutzerfehlern), Skalierbarkeit
 an die RAM-Grenze und darüber hinaus, höchste Perfor-
 mance für unsere Datenmengen+gute System Ressourcen.

-> für kleine Datenmengen macht chunking keinen Sinn.

Dateitypen:

Beide Anwendungen lassen sich in unter 20 Z. mit Arrow
 programmieren (verständlich lesbar) s. Opencode Repo.

RAM Wachstum Arrow, Polars, DuckDB, Data.table,
 readr, baser mit und ohne sel.

512 im FDZ Obergrenze.

Gewinne ggü. Klassischen Methoden 7.X Min DT und
 sehr viel RAM im Default. Dennoch nicht komplett davon
 abraten.

5.6 Fazit (nach Hinten)

- > Empfehlung (insb. für unerfahrene Nutzende): Ar-
row, stabil, flexibel, einfach zu nutzen
- > Profi's in **data.table**, die sehr vertraut sind mit der
Syntax, müssen nur in Randfällen wechseln. Falls die
wirklich genutzten Variablenauswahl nicht durch den

verfügbaren Arbeitsspeicher bedient werden können
 (dt brauchte ca. 3x RAM ggü. Festplatte).

- > neuste, beste, experimentell, bietet **polars** für mod-
erate Datenmengen die effizienteste Variante
- > wer mit sql vertraut ist oder Features außerhalb des
Scopes von Arrow braucht (auf großen Inputs), nutzt
auch gerne duckdb

5.7 Anhang

Weitere Benchmarks

->

Tabelle 5

Benchmarkergebnisse für Fallzahlen (kleine Stichproben)

Beobachtungen	Paket	Dateiformat	Optimierung	Arbeitsspeicher	Laufzeit	Festplattenspeicher
100.000	{polars}	Parquet	-	3.17M	967.38ms	4.59G
100.000	{duckdb}	Parquet	-	4.56M	1.46s	4.59G
100.000	{arrow}	Parquet	Variablenauswahl	261M	1.69s	4.59G
100.000	{data.table}	CSV	Variablenauswahl	40.94M	2.51s	12.11G
100.000	{arrow}	Parquet	Datenaufteilung (gleichmäßig)	219.08M	2.81s	4.54G
100.000	{arrow}	Parquet	-	260.2M	2.84s	4.59G
100.000	{arrow}	Parquet	-	219.7M	3.81s	4.59G
100.000	{arrow}	Parquet	Datenaufteilung (Berichtsjahr)	219.25M	3.85s	4.48G
100.000	{data.table}	CSV	-	31.41G	7.4s	12.11G
100.000	{data.table}	CSV	Datenaufteilung (Berichtsjahr)	46.98G	13.62s	9.74G
10.000	{polars}	Parquet	-	3.13M	404.58ms	469.79M
10.000	{data.table}	CSV	Variablenauswahl	4.2M	669.04ms	1.21G
10.000	{arrow}	Parquet	Variablenauswahl	220.03M	1.1s	469.79M
10.000	{readr}	CSV	Variablenauswahl	8.85M	1.25s	1.21G
10.000	{arrow}	Parquet	-	219.19M	1.59s	469.79M
10.000	{duckdb}	Parquet	-	4.49M	2.94s	469.79M
10.000	{data.table}	CSV	-	3.15G	3.65s	1.21G
10.000	{arrow}	Parquet	Datenaufteilung (Berichtsjahr)	219.23M	4.18s	464.38M
10.000	{arrow}	Parquet	-	219.7M	4.56s	469.79M
10.000	{data.table}	CSV	Datenaufteilung (Berichtsjahr)	4.71G	4.75s	997.06M
10.000	{arrow}	Parquet	Datenaufteilung (gleichmäßig)	219.23M	7.14s	461.09M
10.000	{readr}	CSV	-	1.62G	8.59s	1.21G
10.000	{arrow}	CSV	-	26.1M	15.65s	1.21G
10.000	{data.table}	CSV.GZ	-	4.38G	21.21s	402.31M
10.000	{base}	CSV	-	9.02G	4.86m	1.21G
1.000	{data.table}	CSV	Variablenauswahl	539.57K	124.89ms	123.83M
1.000	{readr}	CSV	Variablenauswahl	2.54M	208.72ms	123.83M
1.000	{polars}	Parquet	-	3.13M	436.83ms	45.87M
1.000	{data.table}	CSV	-	325.99M	900.98ms	123.83M
1.000	{duckdb}	Parquet	-	4.49M	1.09s	45.87M
1.000	{arrow}	Parquet	Variablenauswahl	215.77M	1.21s	45.87M
1.000	{arrow}	Parquet	-	214.86M	1.45s	45.87M
1.000	{arrow}	CSV	-	3.3M	1.51s	123.83M
1.000	{data.table}	CSV.GZ	-	468.94M	2.71s	40.11M
1.000	{data.table}	CSV	Datenaufteilung (Berichtsjahr)	487.24M	2.78s	99.63M
1.000	{arrow}	Parquet	Datenaufteilung (Berichtsjahr)	219.25M	2.94s	51.03M
1.000	{arrow}	Parquet	-	219.7M	3.66s	45.87M
1.000	{arrow}	Parquet	Datenaufteilung (gleichmäßig)	219.1M	3.99s	48.12M
1.000	{readr}	CSV	-	188.25M	4.9s	123.83M
1.000	{base}	CSV	-	680.31M	23.33s	123.83M

Tabelle 6
Benchmarkergebnisse für Regression (kleine Stichproben)

Beobachtungen	Paket	Dateiformat	Optimierung	Arbeitsspeicher	Laufzeit	Festplattenspeicher
100.000	{arrow}	Parquet	Datenaufteilung (Berichtsjahr)	505.21M	3.8s	4.48G
100.000	{dplyr}	CSV	Variablenauswahl	147.61M	13.21s	12.11G
10.000	{dplyr}	CSV	Variablenauswahl	16.79M	1.34s	1.21G
10.000	{arrow}	Parquet	Datenaufteilung (Berichtsjahr)	438.01M	4.96s	464.38M
10.000	{dplyr}	CSV	-	1.7G	10.82s	1.21G
1.000	{dplyr}	CSV	Variablenauswahl	3.97M	259.24ms	123.83M
1.000	{dplyr}	CSV	-	197.32M	6.42s	123.83M
1.000	{arrow}	Parquet	Datenaufteilung (Berichtsjahr)	431.53M	6.84s	51.03M